



Swiss
Paraplegic
Research

Data Visualization

Workshop R-Introduction

Yannick Rothacher, Data Scientist

13.10.2026



Data visualization in R

- The **visualization of data** is a fundamental part of data analysis
- Data visualizations are often the best way to **convey complex information**
- There are many functions in R to create different plots
 - We will first focus on the basic **plot** function...
 - ...illustrated on this simple data set:

> d

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

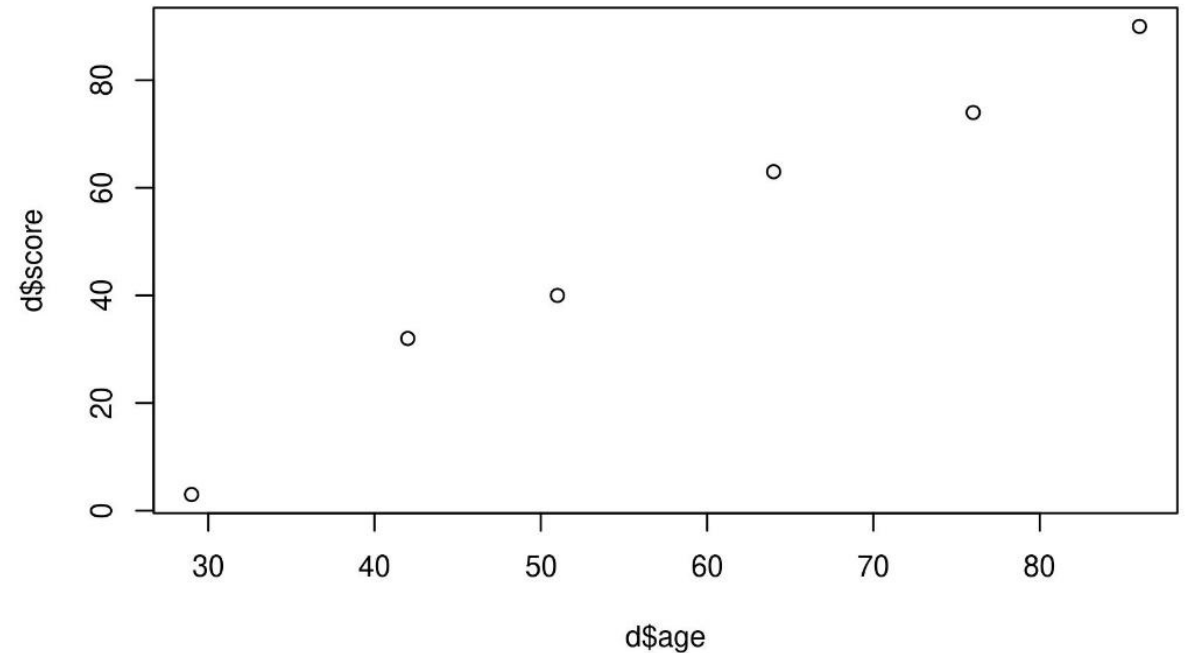
The plot function

- Using **plot** to create a scatter plot:

```
> d
```

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

```
> plot(x = d$age, y = d$score)
```



The plot function

- Using **plot** to create a scatter plot:

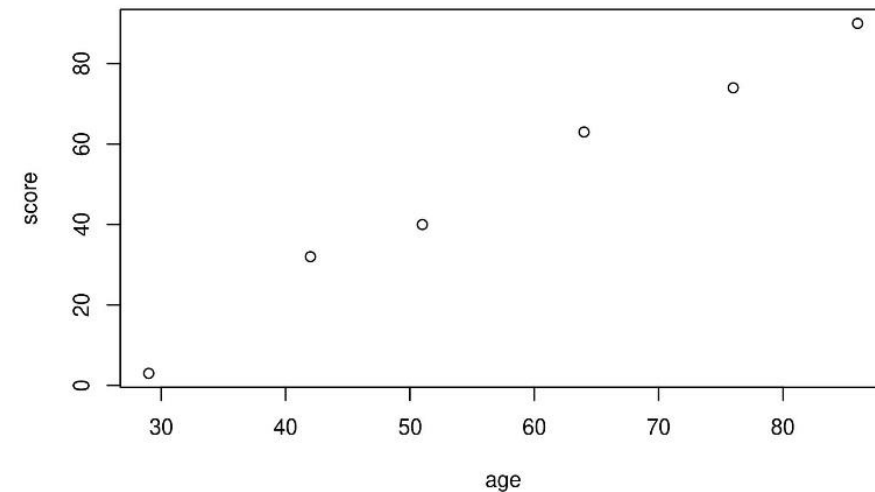
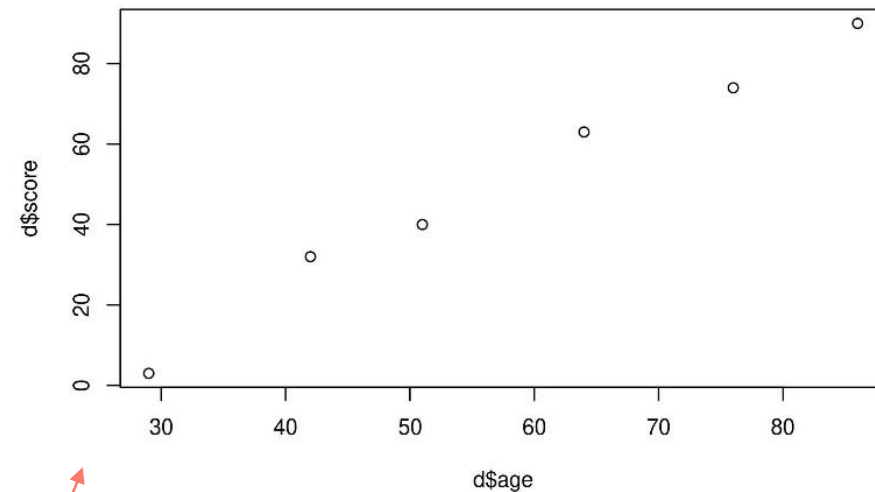
```
> d
```

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

```
> plot(x = d$age, y = d$score)
```

Using a formula:

```
> plot(score ~ age, data = d)
```



The plot function

-> How the plot function processes the data:

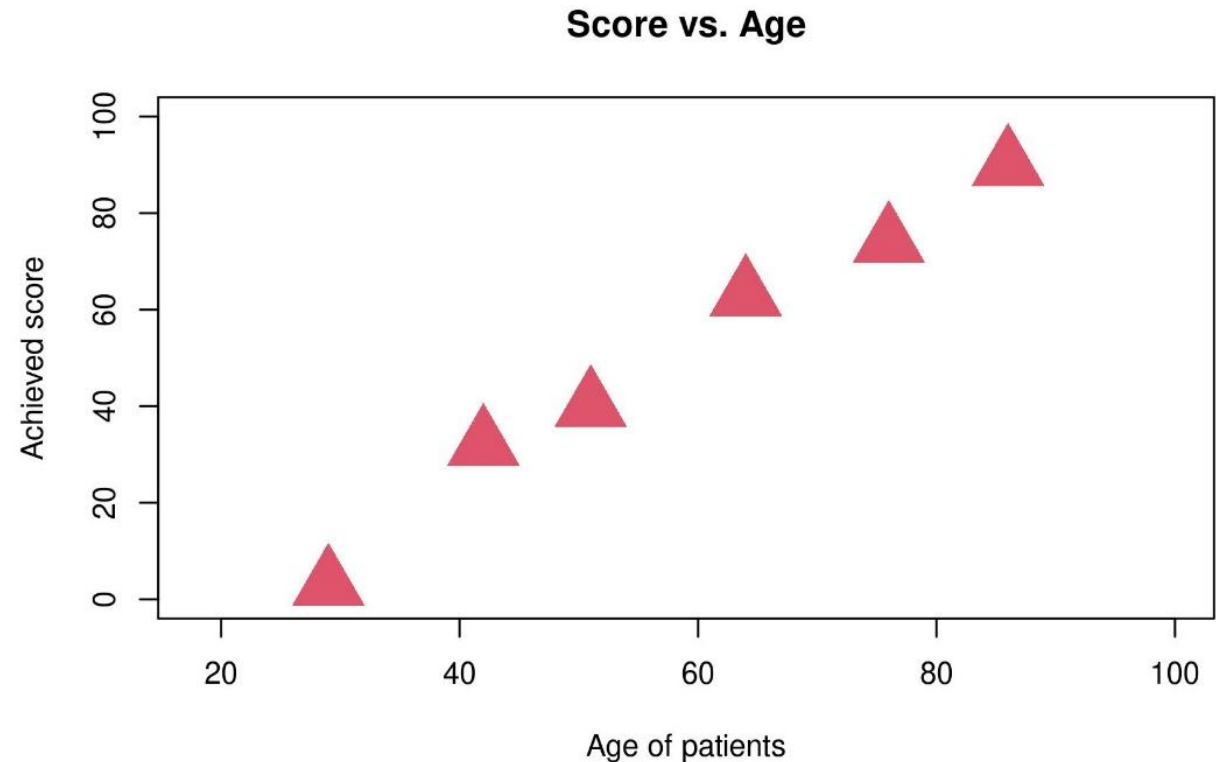
- **plot** can take many additional arguments:

> d

```
      id   SCIstatus   group  score  age
1 pat1  paraplegic   cntrl     3    29
2 pat2  tetraplegic  treatm    32    42
3 pat3  tetraplegic   cntrl    40    51
4 pat4  paraplegic   treatm    63    64
5 pat5  paraplegic   cntrl    74    76
6 pat6  tetraplegic  treatm    90    86
```

x	y	pch	col	cex
29	3	17	2	4
42	32	17	2	4
51	40	17	2	4
64	63	17	2	4
76	74	17	2	4
86	90	17	2	4

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 17,
      col = 2,
      cex = 4)
```



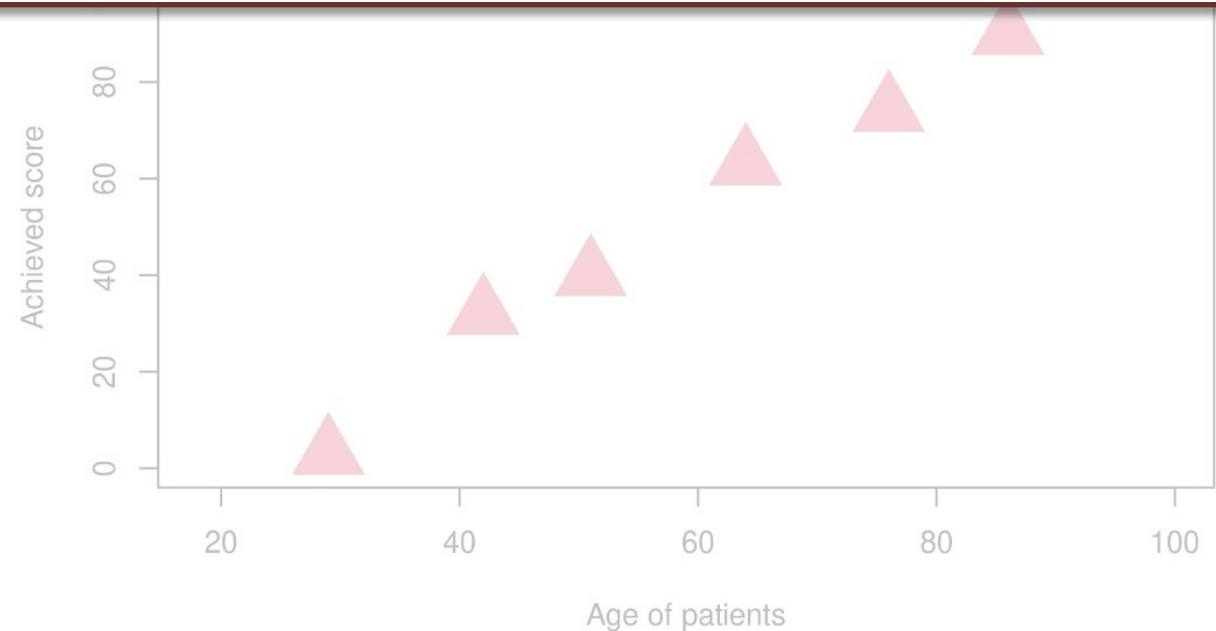
The plot function

- **plot** can take many additional arguments

```
> d
  id   SCIstatus   group  score  age
1 pat1 paraplegic  cntrl    3    29
2 pat2 tetraplegic treatm   32    42
3 pat3 tetraplegic  cntrl   40    51
4 pat4 paraplegic  treatm   63    64
5 pat5 paraplegic  cntrl   74    76
6 pat6 tetraplegic treatm   90    86
```

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 17,
      col = 2,
      cex = 4)
```

The **pch** argument defines the **shape** of the points:



The `plot` function

- `plot` can take many additional arguments

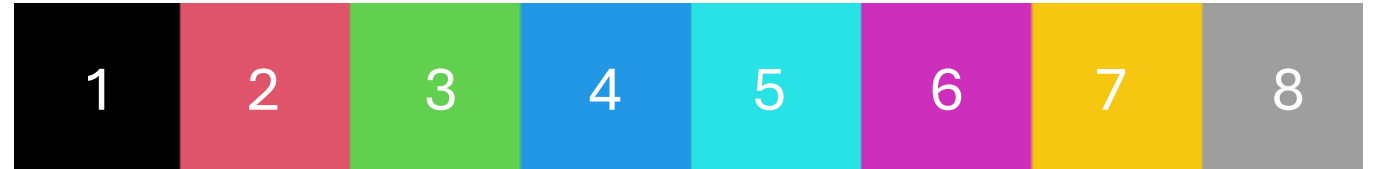
> d

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 17,
      col = 2, ←
      cex = 4)
```

The `col` argument defines the **color** of the points

- Can define color through number:



- ...through name:
`col = "green"`
- ...through Hex code:
`col = "#00FF00"`
- ...through rgb values:
`col = rgb(0,1,0)`

20

40

60

80

100

Age of patients



The plot function

-> How the plot function processes the data:

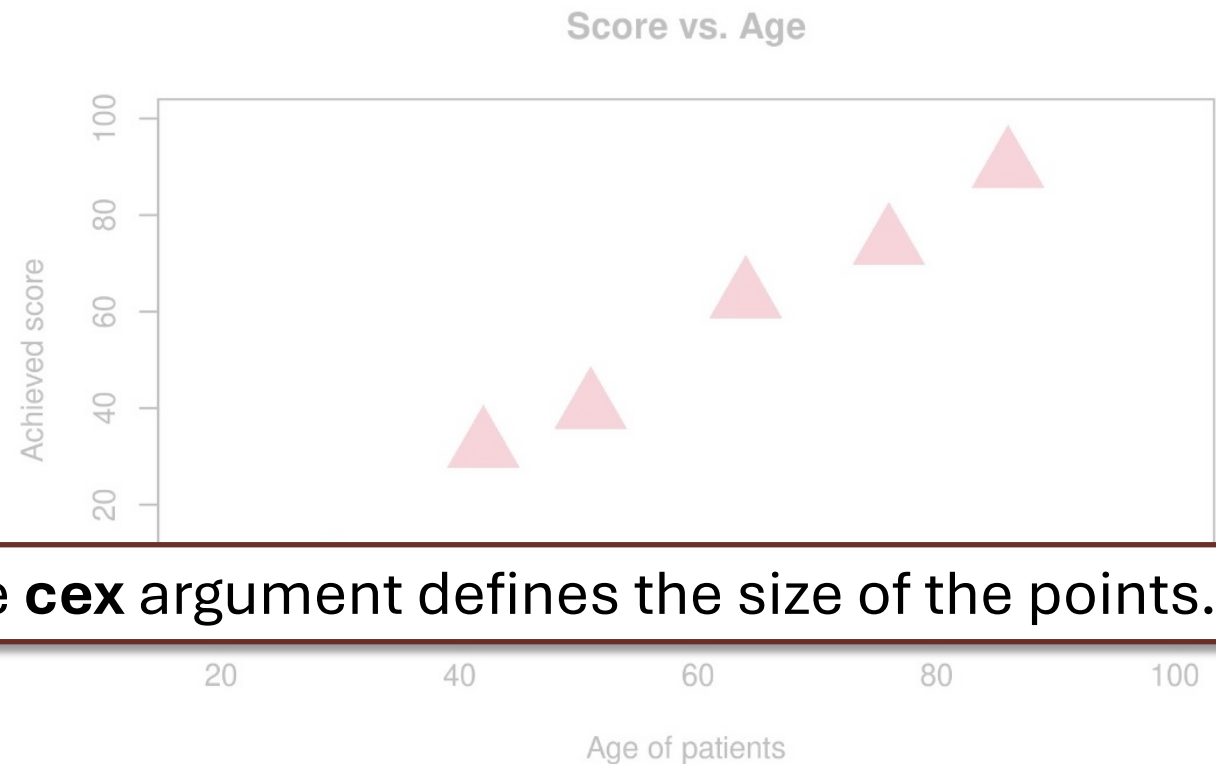
- **plot** can take many additional arguments:

> d

```
      id   SCIstatus   group  score  age
1 pat1  paraplegic   cntrl    3    29
2 pat2  tetraplegic  treatm   32    42
3 pat3  tetraplegic   cntrl   40    51
4 pat4  paraplegic   treatm   63    64
5 pat5  paraplegic   cntrl   74    76
6 pat6  tetraplegic  treatm   90    86
```

x	y	pch	col	cex
29	3	17	2	4
42	32	17	2	4
51	40	17	2	4
64	63	17	2	4
76	74	17	2	4
86	90	17	2	4

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 17,
      col = 2,
      cex = 4)
```



The **cex** argument defines the size of the points.

The plot function

- **plot** can take many additional arguments:

> d

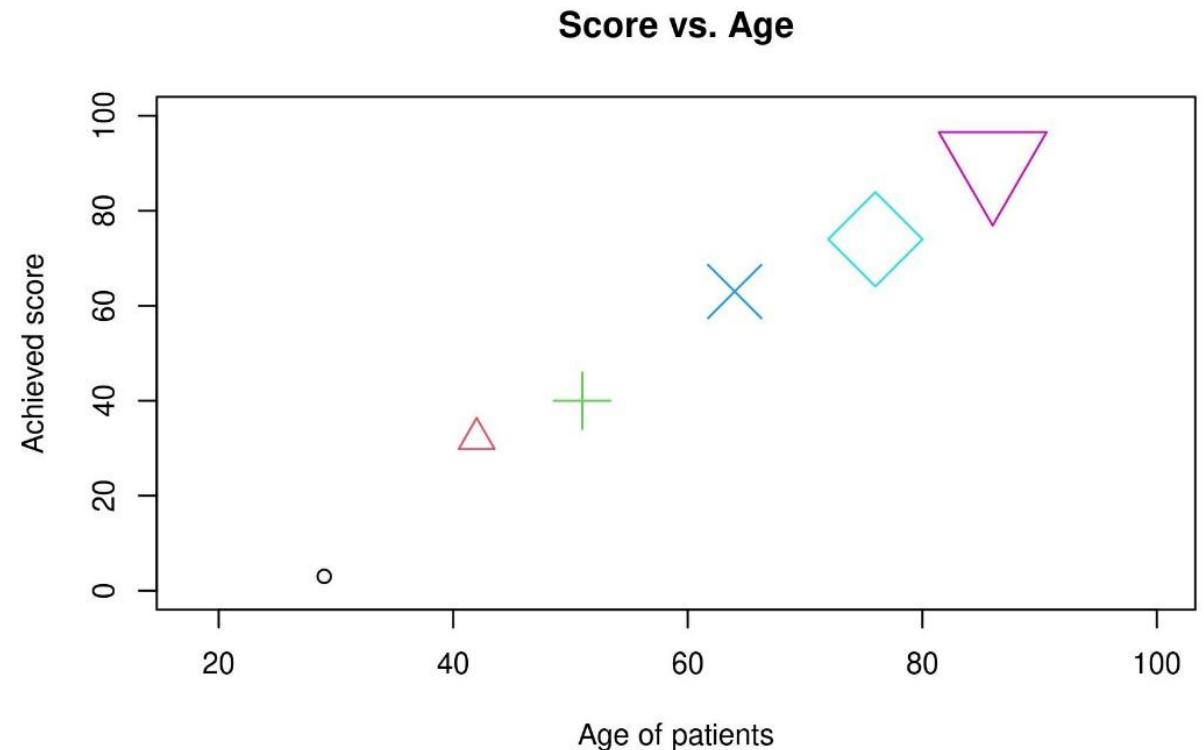
```
      id  SCIstatus  group  score  age
1 pat1  paraplegic  cntrl     3   29
2 pat2  tetraplegic  treatm    32   42
3 pat3  tetraplegic  cntrl    40   51
4 pat4  paraplegic  treatm    63   64
5 pat5  paraplegic  cntrl    74   76
6 pat6  tetraplegic  treatm    90   86
```

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 1:nrow(d),
      col = 1:nrow(d),
      cex = 1:nrow(d))
```

Supplied as
vectors

-> How the plot function processes the data:

x	y	pch	col	cex
29	3	1	1	1
42	32	2	2	2
51	40	3	3	3
64	63	4	4	4
76	74	5	5	5
86	90	6	6	6



The plot function

- **plot** can take many additional arguments:

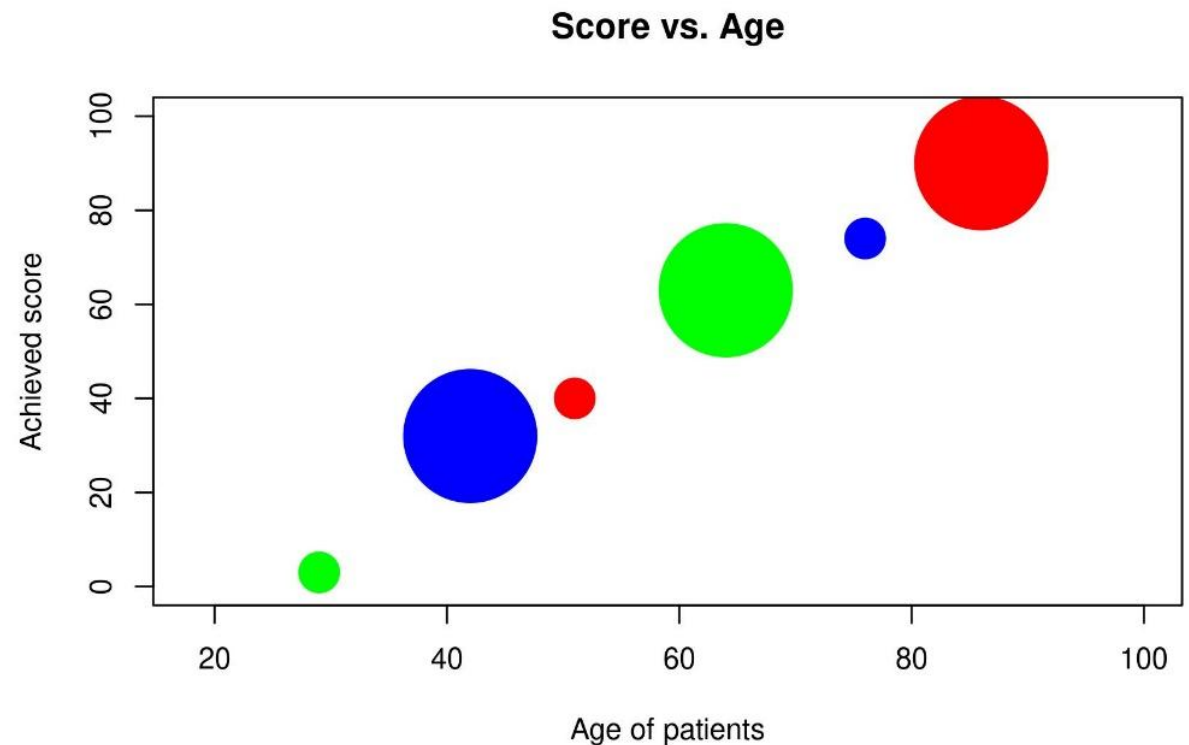
> d

```
  id  SCIstatus  group  score  age
1 pat1 paraplegic  cntrl    3   29
2 pat2 tetraplegic treatm   32   42
3 pat3 tetraplegic  cntrl   40   51
4 pat4 paraplegic  treatm   63   64
5 pat5 paraplegic  cntrl   74   76
6 pat6 tetraplegic treatm   90   86
```

```
> plot(x = d$age, y = d$score,
       main = "Score vs. Age",
       xlab = "Age of patients",
       ylab = "Achieved score",
       xlim = c(18, 100),
       ylim = c(0, 100),
       pch = 19,
       col = c("green", "blue", "red"),
       cex = c(3, 10))
```

-> How the plot function processes the data:

x	y	pch	col	cex
29	3	19	green	3
42	32	19	blue	10
51	40	19	red	3
64	63	19	green	10
76	74	19	blue	3
86	90	19	red	10



The plot function

- **plot** can take many additional arguments:

```
> d
  id   SCIstatus   group  score  age
1 pat1 paraplegic  cntrl    3   29
2 pat2 tetraplegic treatm   32   42
3 pat3 tetraplegic  cntrl   40   51
4 pat4 paraplegic  treatm   63   64
5 pat5 paraplegic  cntrl   74   76
6 pat6 tetraplegic  treatm   90   86
```

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 19,
      col = d$SCIstatus,
      cex = c(3, 10))

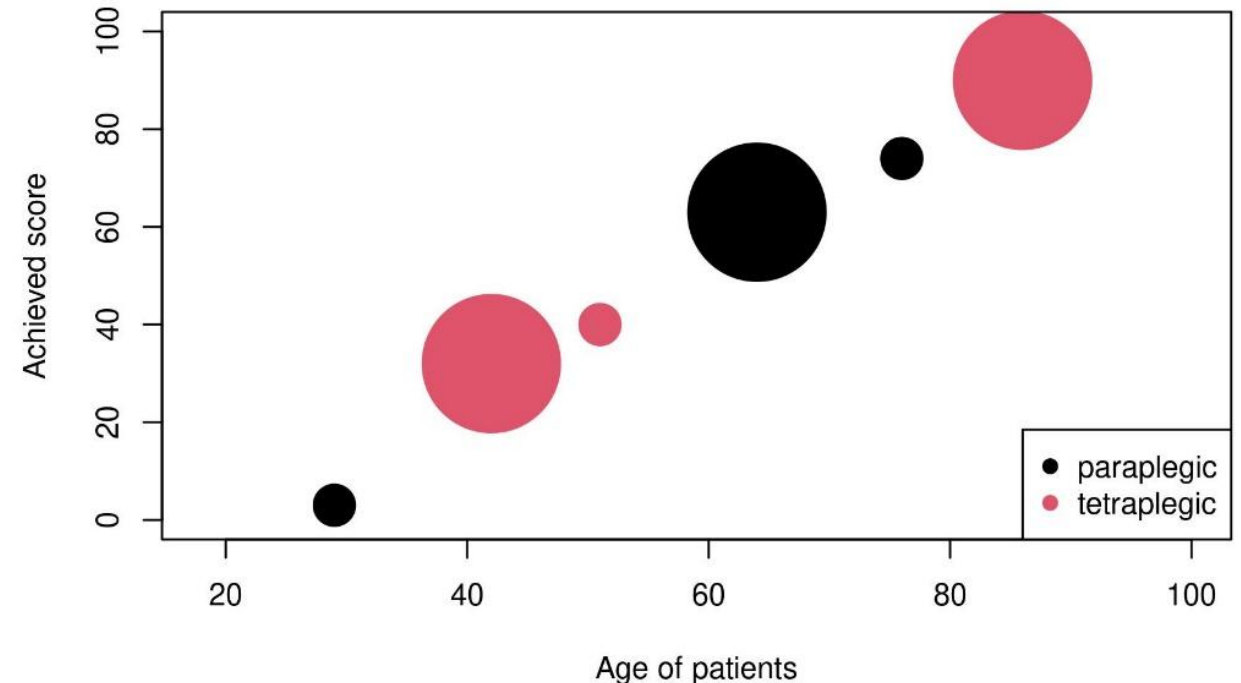
Supplied as a factor

> legend("bottomright", legend=levels(d$SCIstatus),
      col=1:nlevels(d$SCIstatus), pch=19)
```

-> How the plot function processes the data:

x	y	pch	col	cex
29	3	19	1	3
42	32	19	2	10
51	40	19	2	3
64	63	19	1	10
76	74	19	1	3
86	90	19	2	10

Score vs. Age



The plot function

- **plot** can take many additional arguments:

```
> d
  id   SCIstatus   group  score  age
1 pat1 paraplegic  cntrl    3   29
2 pat2 tetraplegic treatm   32   42
3 pat3 tetraplegic  cntrl   40   51
4 pat4 paraplegic  treatm   63   64
5 pat5 paraplegic  cntrl   74   76
6 pat6 tetraplegic  treatm   90   86
```

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      pch = 19,
      col = d$SCIstatus,
      cex = c(3, 10))
```

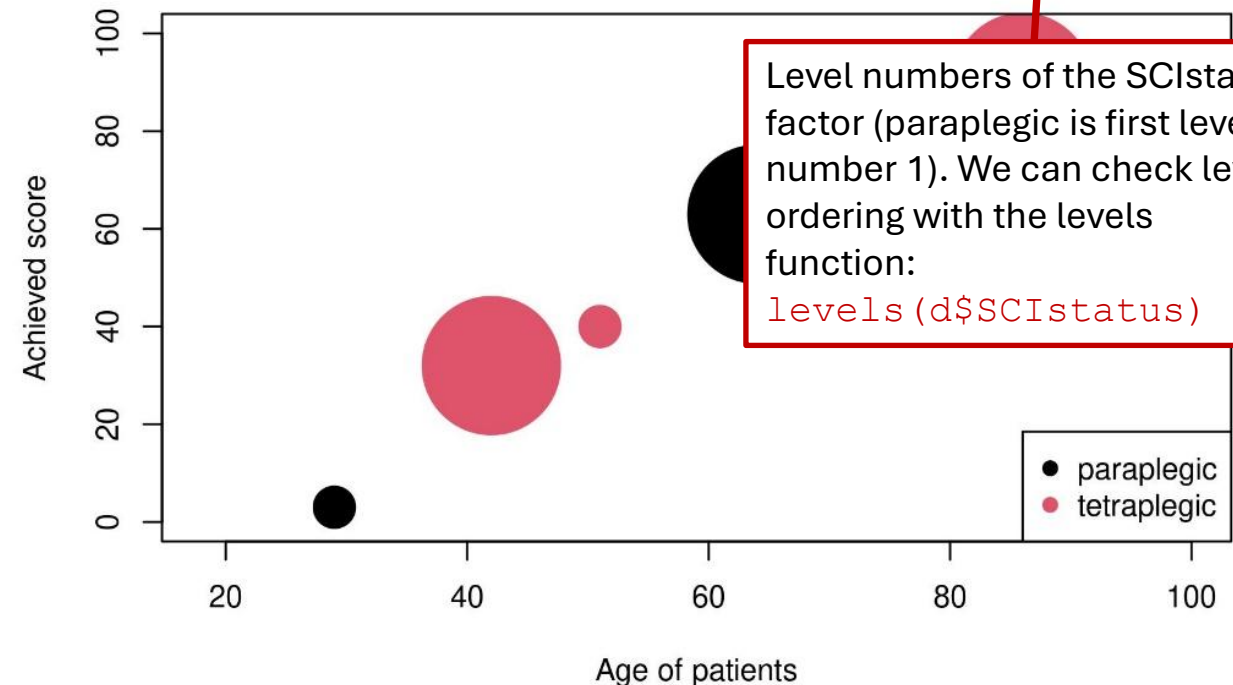
Supplied as a
factor

```
> legend("bottomright", legend=levels(d$SCIstatus),
      col=1:nlevels(d$SCIstatus), pch=19)
```

-> How the plot function processes the data:

x	y	pch	col	cex
29	3	19	1	3
42	32	19	2	10
51	40	19	2	3
64	63	19	1	10
76	74	19	1	3
86	90	19	2	10

Score vs. Age

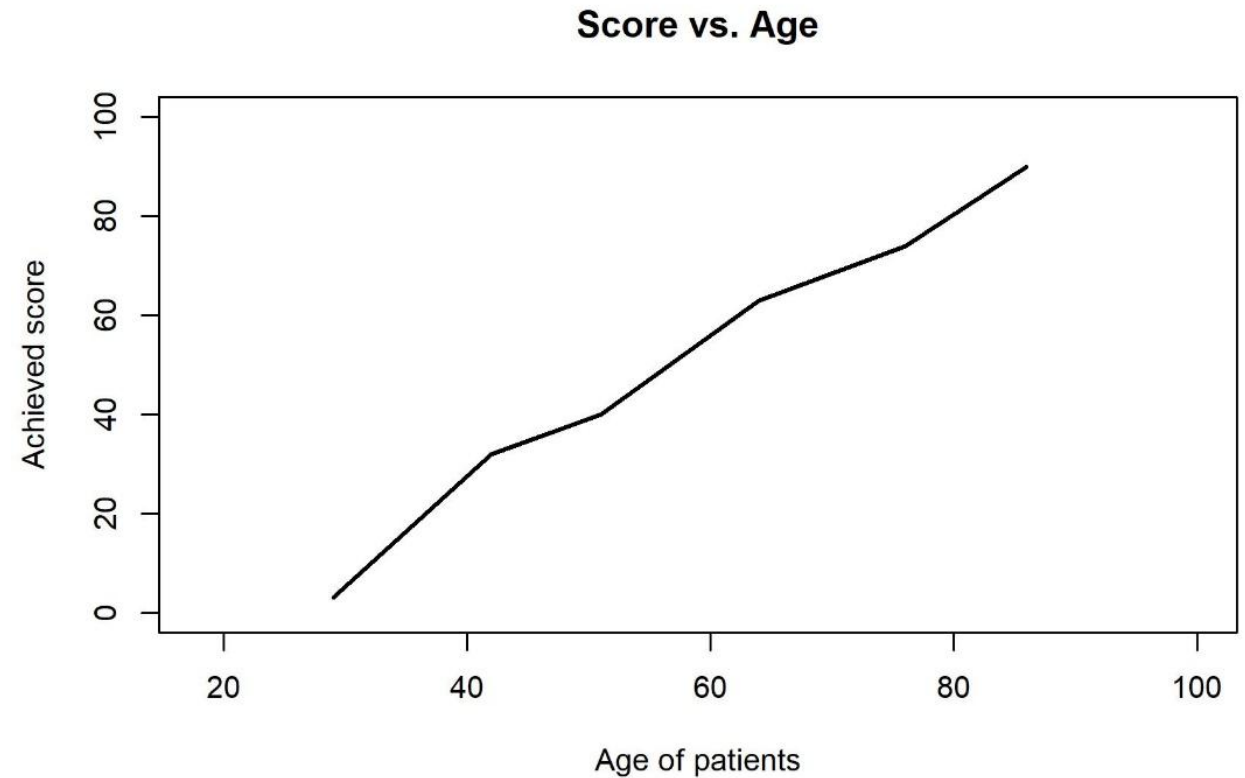


Plotting lines

- We can plot lines instead of individual points:

```
> d
  id  SCIstatus  group  score  age
1 pat1 paraplegic  cntrl    3   29
2 pat2 tetraplegic treatm   32   42
3 pat3 tetraplegic  cntrl   40   51
4 pat4 paraplegic  treatm   63   64
5 pat5 paraplegic  cntrl   74   76
6 pat6 tetraplegic treatm   90   86
```

```
> plot(x = d$age, y = d$score,
       main = "Score vs. Age",
       xlab = "Age of patients",
       ylab = "Achieved score",
       xlim = c(18, 100),
       ylim = c(0, 100),
       type = "l",
       lty = 1,
       lwd = 3)
```



Plotting lines

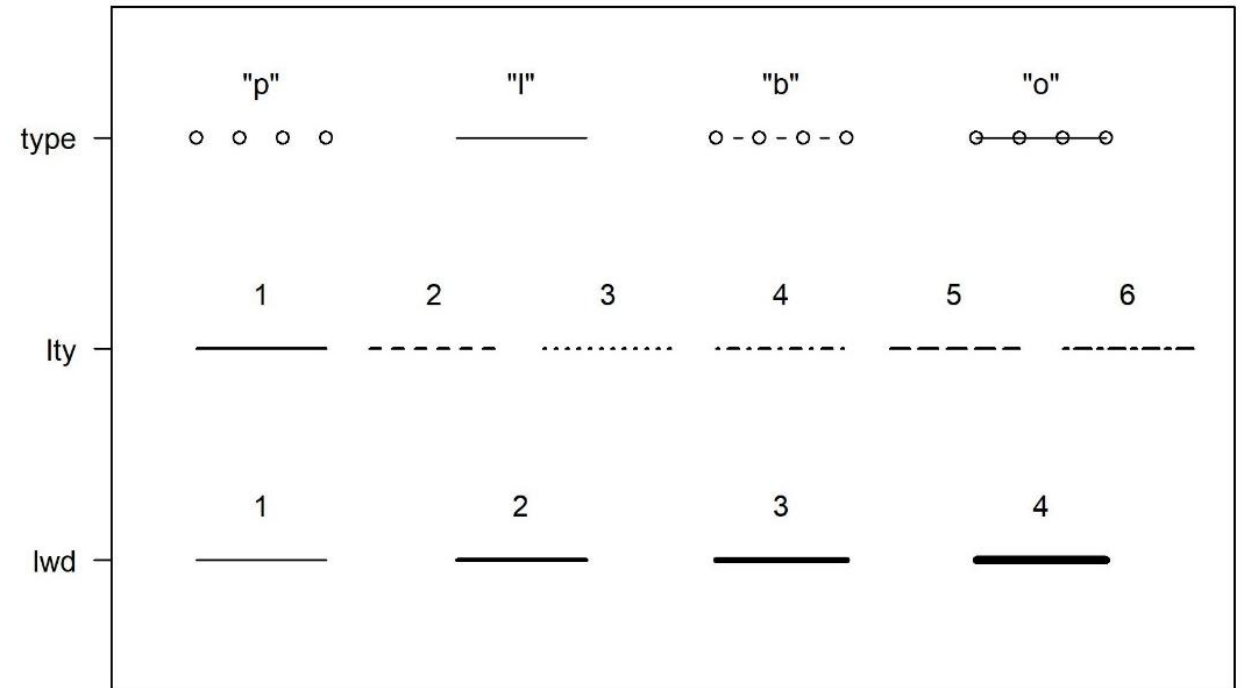
- We can plot lines instead of individual points:

```
> d
  id   SCIstatus   group  score  age
1 pat1 paraplegic  cntrl    3   29
2 pat2 tetraplegic treatm   32   42
3 pat3 tetraplegic  cntrl   40   51
4 pat4 paraplegic  treatm   63   64
5 pat5 paraplegic  cntrl   74   76
6 pat6 tetraplegic  treatm   90   86
```

```
> plot(x = d$age, y = d$score,
      main = "Score vs. Age",
      xlab = "Age of patients",
      ylab = "Achieved score",
      xlim = c(18, 100),
      ylim = c(0, 100),
      type = "l",
      lty = 1,
      lwd = 3)
```

Score vs. Age

The **look of a line** can be customized with various arguments, for example:



Adding points or lines to existing plot

- We can **add points** to an existing plot using the **points** function
- We assume we have an existing plot

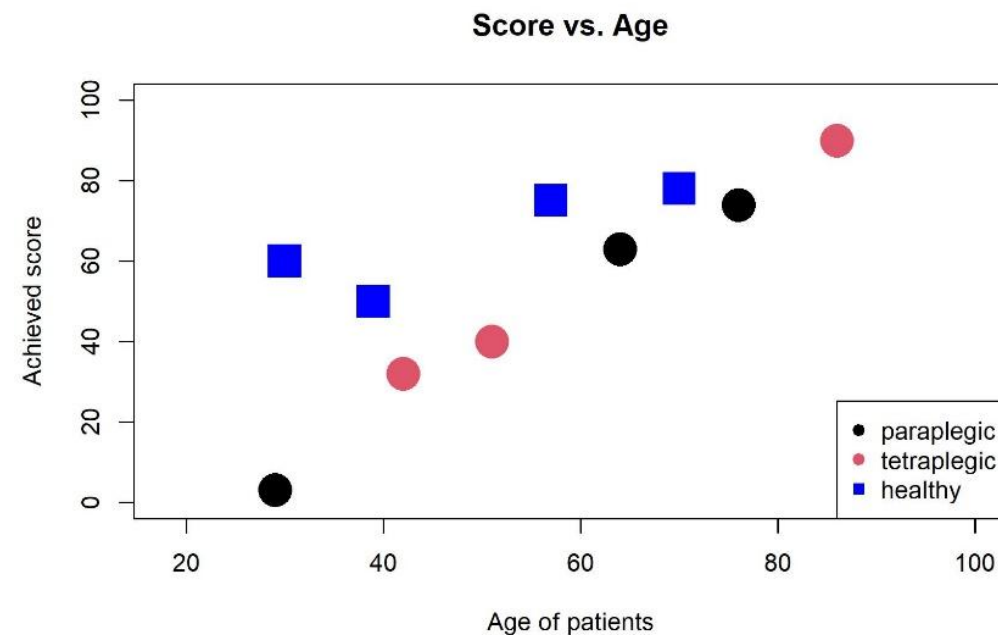
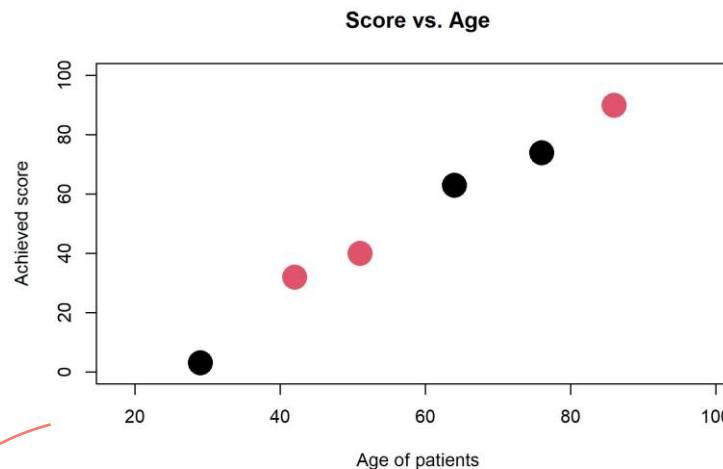
- And some new data:

```
> dhealth
```

	Id	SCIstatus	score	age
1	pers1	healthy	60	30
4	pers4	healthy	50	39
2	pers2	healthy	75	57
3	pers3	healthy	78	70

```
> points(x = dhealth$age,  
         y = dhealth$score,  
         col = "blue",  
         cex = 3,  
         pch = 15)
```

```
> legend(...)
```



Adding points or lines to existing plot

- We can **add lines** to an existing plot using the **lines** function
- We assume we have an existing plot

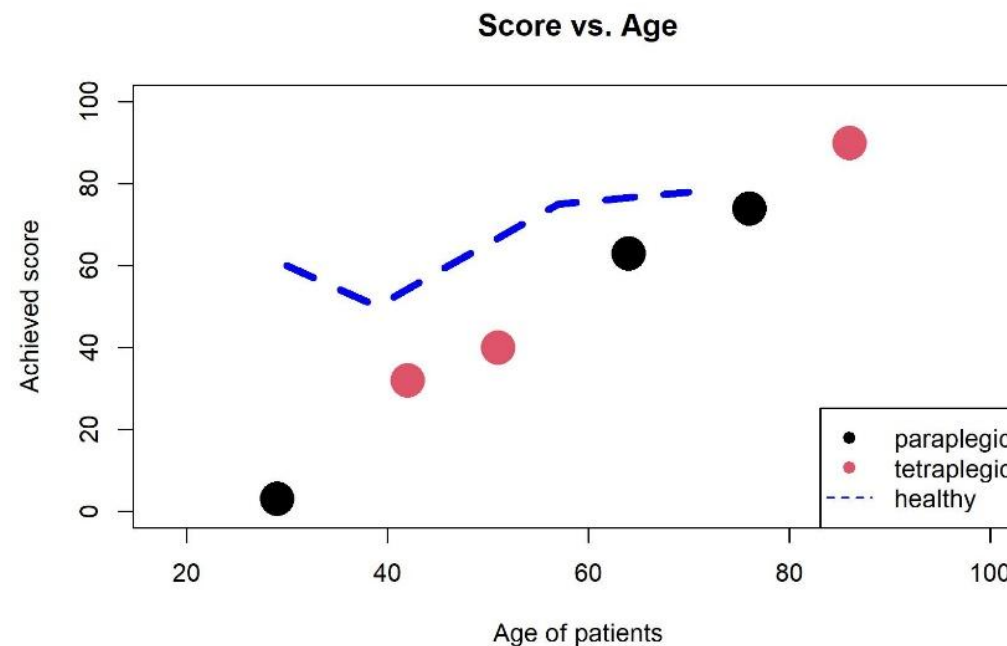
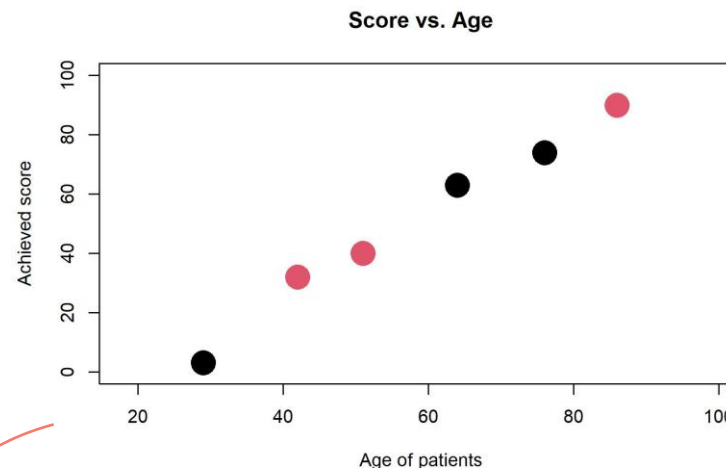
- And some new data:

> **dhealth**

	Id	SCIstatus	score	age
1	pers1	healthy	60	30
4	pers4	healthy	50	39
2	pers2	healthy	75	57
3	pers3	healthy	78	70

```
> lines(x = dhealth$age,  
        y = dhealth$score,  
        col = "blue",  
        lty = 2,  
        lwd = 4)
```

```
> legend(...)
```



Plotting a categorical variable (barplot and pie)

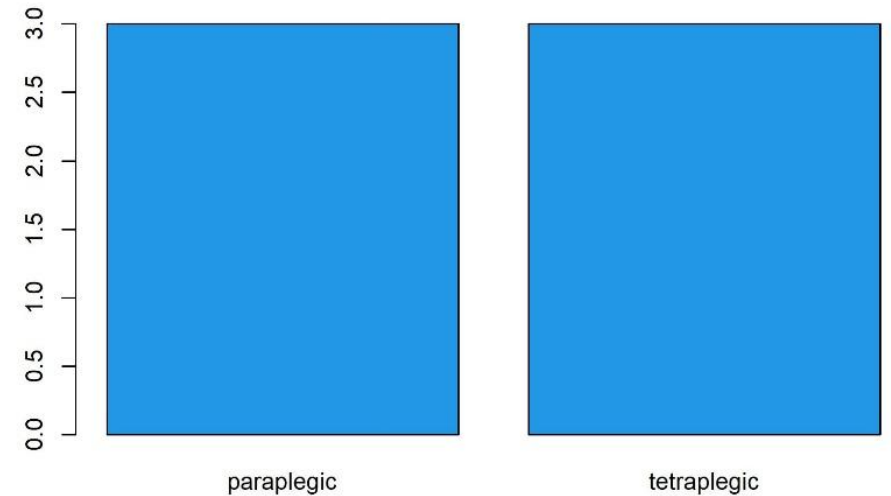
- We can create a barplot to visualize how often the levels of a categorical variable were observed:

```
> d
```

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

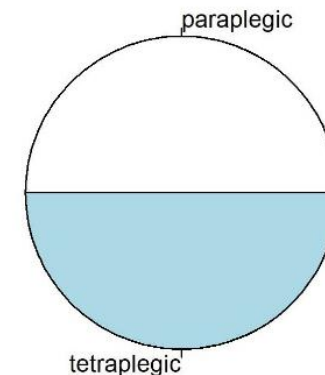
```
> tbl <- table(d$SCIstatus)
```

```
> barplot(tbl, col=4)
```



- An alternative visualization is the pie chart:

```
> pie(tbl)
```



Plotting two categorical variables (barplot)

- We can create a barplot to visualize two categorical variables:

```
> d
```

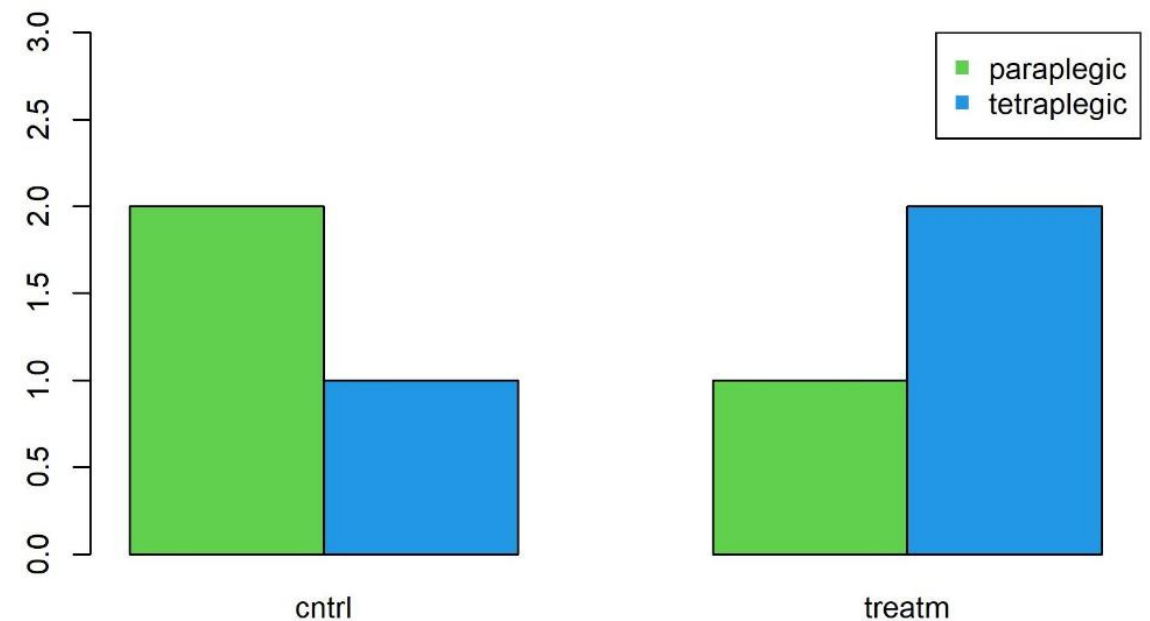
	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

```
> tbl <- table(d$SCIstatus, d$group)
```

```
> cls <- c(3,4)
```

```
> barplot(tbl, beside = TRUE, col = cls,  
          ylim = c(0, 3))
```

```
> legend("topright",  
        legend = levels(d$SCIstatus),  
        col = cls, pch=15)
```



Plotting a metric vs a categorical variable (boxplot)

- Plotting a metric variable vs a categorical variable (coded as a **factor**):

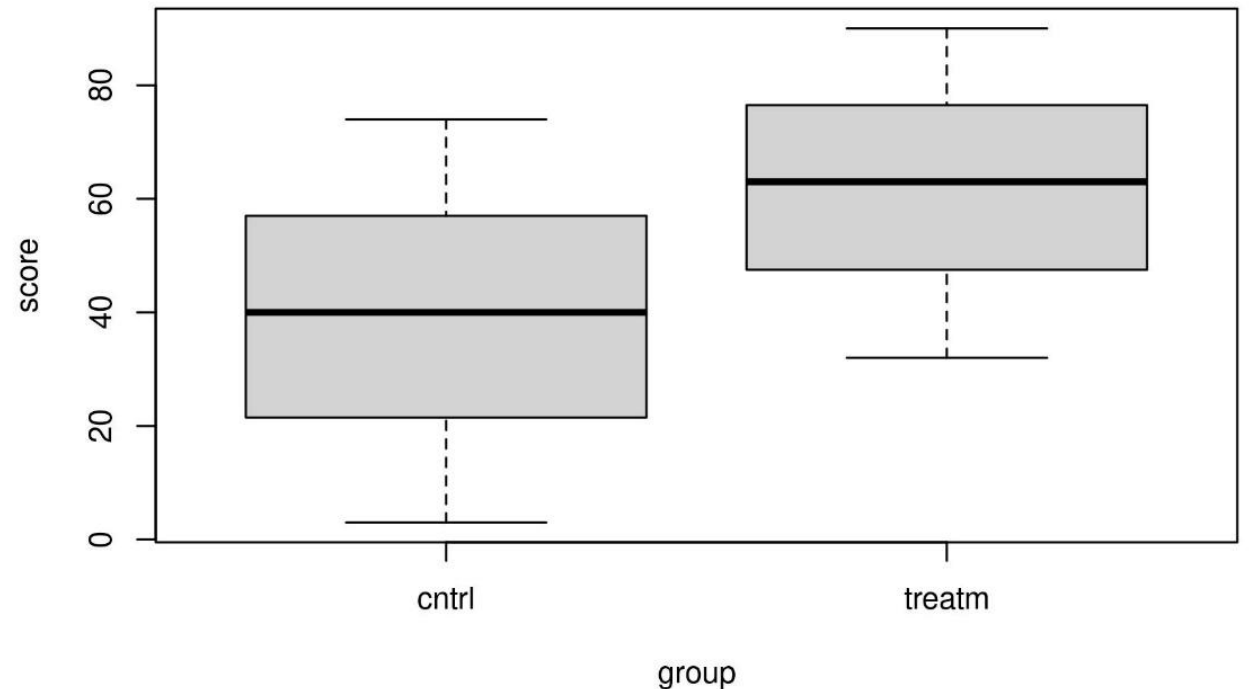
> **d**

	id	SCIstatus	group	score	age
1	pat1	paraplegic	cntrl	3	29
2	pat2	tetraplegic	treatm	32	42
3	pat3	tetraplegic	cntrl	40	51
4	pat4	paraplegic	treatm	63	64
5	pat5	paraplegic	cntrl	74	76
6	pat6	tetraplegic	treatm	90	86

> **plot(score ~ group, data = d)**

- The plot function internally calls the `boxplot` function, we can also create the same plot using `boxplot` directly:

> **boxplot(score ~ group, data = d)**



Flexibility of base R plotting

- We've seen multiple examples of base R plotting commands
- One advantage of base R plotting functions is their flexibility, nearly every aspect of a figure can be customized, e.g.
 - Control of symbols, lines and shapes
 - Appearance can be customized, including axes, titles and layout
 - Text can be added to figures
 - ...
- As an alternative, there are **specialized plotting packages**
 - These package are supposed to facilitate the generation of certain visualizations
 - We will look at the example of **ggplot**

Creating figures with ggplot

- **ggplot2** is a very popular package for the creation of (complex) data visualizations
- We will look at its application using this exemplary data frame:

```
> d1rg
  id   SCIstatus  group      age  score
1  pat1 tetraplegic cntrl   oldAdults  2.86
2  pat2 paraplegic treatm youngAdults 20.89
3  pat3 paraplegic cntrl   youngAdults 14.00
4  pat4 paraplegic treatm middleAged 18.46
5  pat5 paraplegic cntrl   middleAged  7.53
6  pat6 paraplegic treatm   oldAdults 16.68
7  pat7 tetraplegic cntrl   oldAdults  5.95
8  pat8 paraplegic treatm middleAged 21.18
9  pat9 tetraplegic cntrl   youngAdults  2.96
10 pat10 tetraplegic treatm middleAged  3.26
11 pat11 paraplegic cntrl   middleAged 11.35
12 pat12 tetraplegic treatm   oldAdults  7.81
13 pat13 paraplegic cntrl   middleAged  9.99
```

- Install and load ggplot2 package:

```
> install.packages("ggplot2")
> library(ggplot2)
```

- The syntax of ggplot is different from base plotting functions
 - The commands follow a **layer-by-layer principle**

Creating figures with ggplot

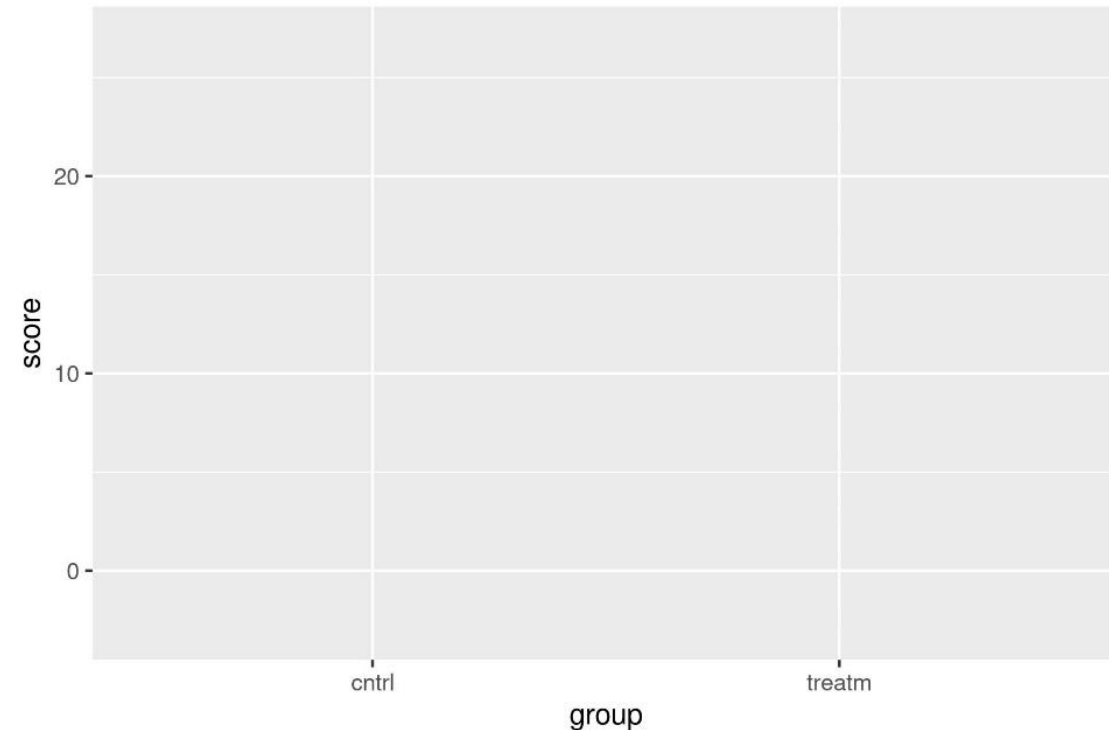
- The commands follow a **layer-by-layer principle**
- The **first layer** is created with the **ggplot** function, it only defines the basic structure of the figure (nothing is yet plotted):

```
> ggplot(data = dlrg,  
         mapping = aes(x = group, y = score))
```

- Two arguments are essential for the **ggplot** function:
 - **data** to pass the data frame
 - **mapping** to define figure structure (using **aes** function)

```
> dlrg
```

	id	SCiStatus	group	age	score
1	pat1	tetraplegic	cntrl	oldAdults	2.86
2	pat2	paraplegic	treatm	youngAdults	20.89
3	pat3	paraplegic	cntrl	youngAdults	14.00
4	pat4	paraplegic	treatm	middleAged	18.46
5	pat5	paraplegic	cntrl	middleAged	7.53



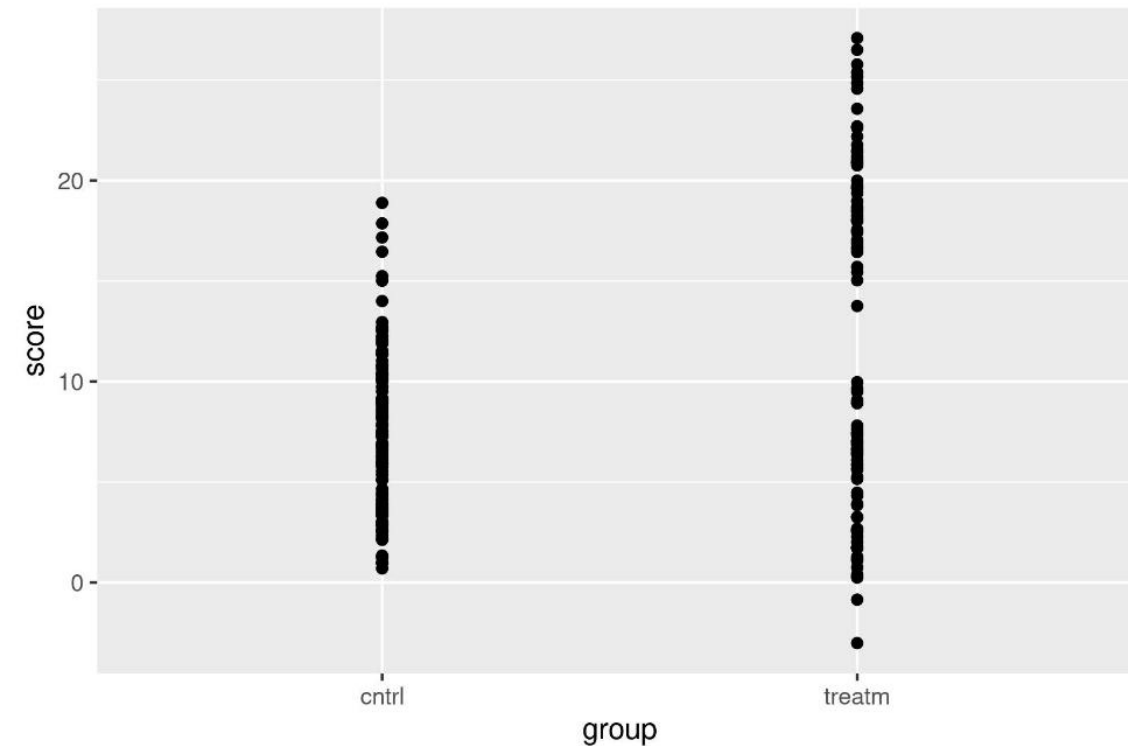
Creating figures with ggplot

- We can add new layers with the **+** sign
- To add **points**, use the **geom_point** function:

```
> ggplot(data = dlrg,  
  mapping = aes(x = group, y = score)) +  
  geom_point()
```

```
> dlrg
```

	id	SCIstatus	group	age	score
1	pat1	tetraplegic	cntrl	oldAdults	2.86
2	pat2	paraplegic	treatm	youngAdults	20.89
3	pat3	paraplegic	cntrl	youngAdults	14.00
4	pat4	paraplegic	treatm	middleAged	18.46
5	pat5	paraplegic	cntrl	middleAged	7.53



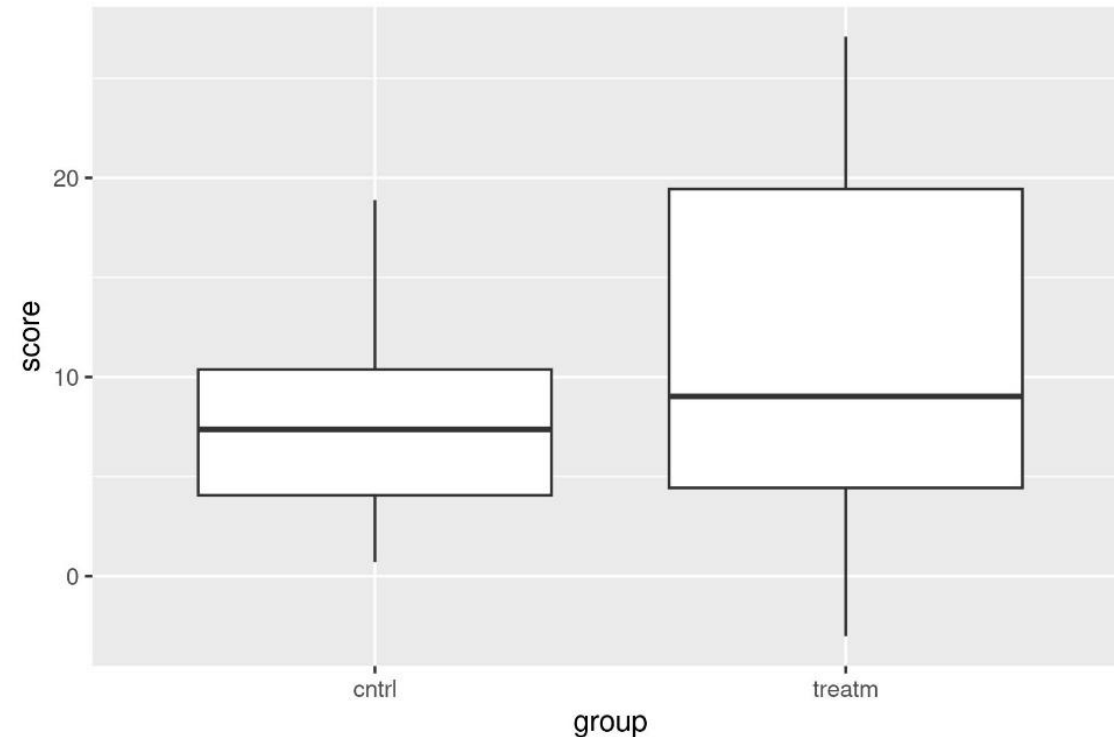
Creating figures with ggplot

- We can add new layers with the **+** sign
- To add **boxplots**, use the **geom_boxplot** function:

```
> ggplot(data = dlrg,  
  mapping = aes(x = group, y = score)) +  
  geom_boxplot()
```

```
> dlrg
```

	id	SCiStatus	group	age	score
1	pat1	tetraplegic	cntrl	oldAdults	2.86
2	pat2	paraplegic	treatm	youngAdults	20.89
3	pat3	paraplegic	cntrl	youngAdults	14.00
4	pat4	paraplegic	treatm	middleAged	18.46
5	pat5	paraplegic	cntrl	middleAged	7.53



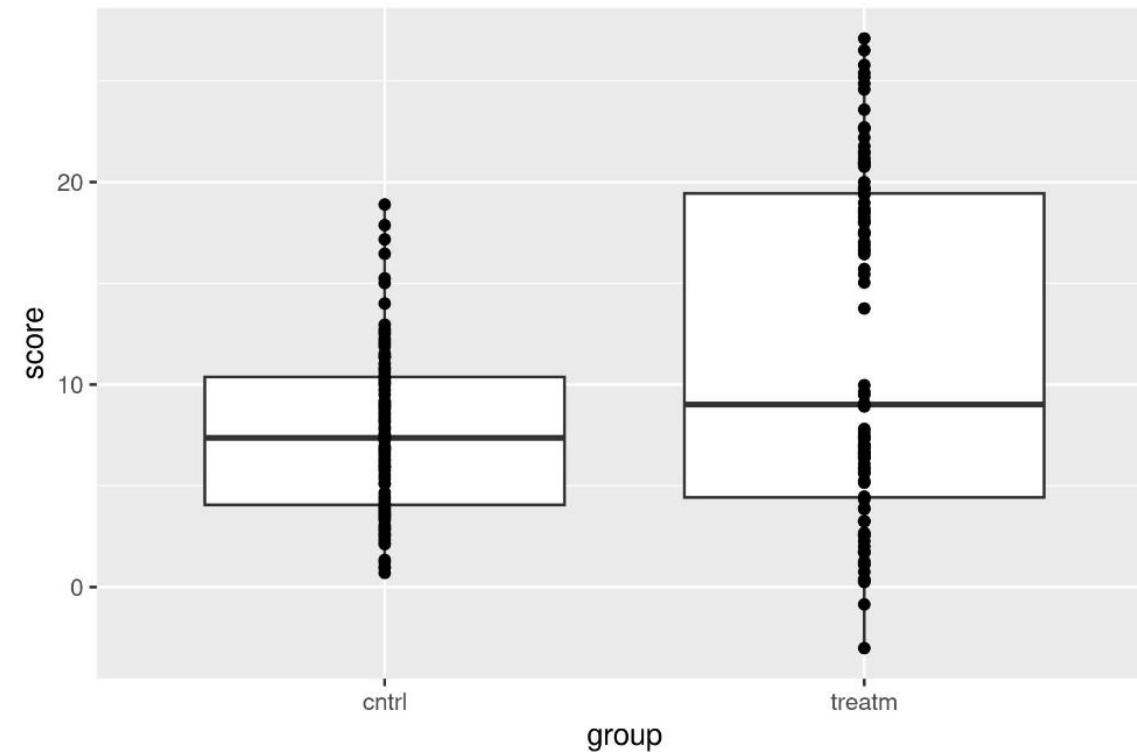
Creating figures with ggplot

- We can add new layers with the **+** sign
- We could also add both layers:

```
> ggplot(data = dlrg,  
  mapping = aes(x = group, y = score)) +  
  geom_boxplot() +  
  geom_point()
```

```
> dlrg
```

	id	SCIstatus	group	age	score
1	pat1	tetraplegic	cntrl	oldAdults	2.86
2	pat2	paraplegic	treatm	youngAdults	20.89
3	pat3	paraplegic	cntrl	youngAdults	14.00
4	pat4	paraplegic	treatm	middleAged	18.46
5	pat5	paraplegic	cntrl	middleAged	7.53

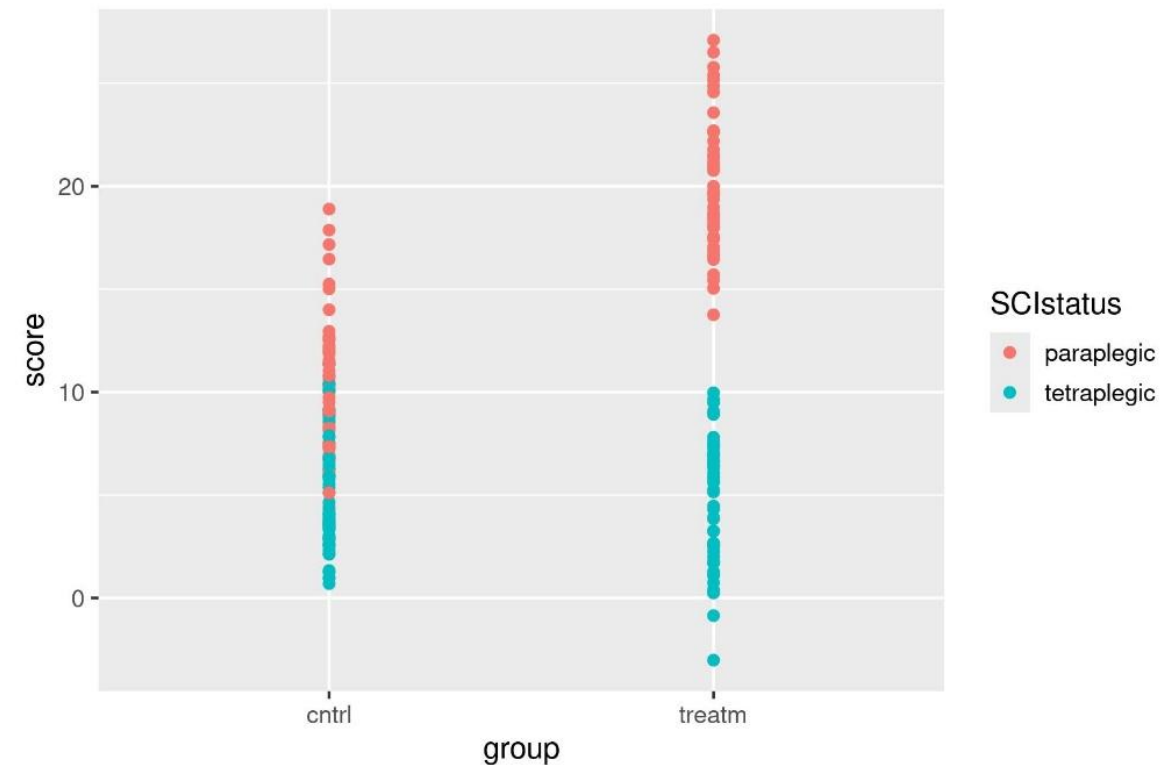


Creating figures with ggplot

- We can add new layers with the **+** sign
- We can change further aesthetics in the **aes** function, e.g. adding colour:

```
> ggplot(data = dlrg,  
  mapping = aes(x = group, y = score,  
                 colour = SCIsstatus)) +  
  geom_point()
```

```
> dlrg  
   id  SCIsstatus  group      age  score  
1  pat1 tetraplegic  cntrl  oldAdults  2.86  
2  pat2 paraplegic  treatm youngAdults 20.89  
3  pat3 paraplegic  cntrl  youngAdults 14.00  
4  pat4 paraplegic  treatm middleAged 18.46  
5  pat5 paraplegic  cntrl  middleAged  7.53
```

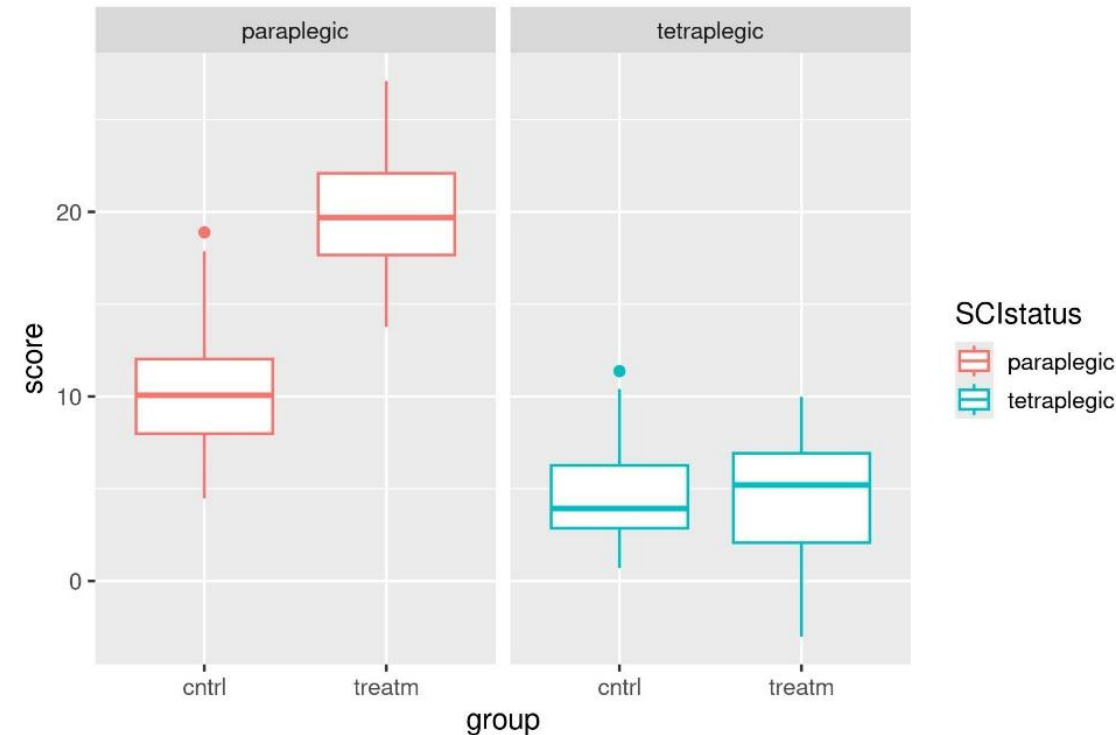


Creating figures with ggplot

- We can add new layers with the **+** sign
- The ggplot function is especially useful for quickly plotting **multiple settings**
 - We can use the **facet_wrap** function for that purpose
 - E.g. splitting the plot along the **group** variable:

```
> ggplot(data = dlrg,  
  mapping = aes(x = group, y = score,  
                 colour = SCiStatus)) +  
  facet_wrap(~ SCiStatus) +  
  geom_boxplot()
```

```
> dlrg  
   id  SCiStatus  group      age  score  
1  pat1 tetraplegic  cntrl  oldAdults  2.86  
2  pat2 paraplegic  treatm youngAdults 20.89  
3  pat3 paraplegic  cntrl  youngAdults 14.00  
4  pat4 paraplegic  treatm middleAged 18.46  
5  pat5 paraplegic  cntrl  middleAged  7.53
```



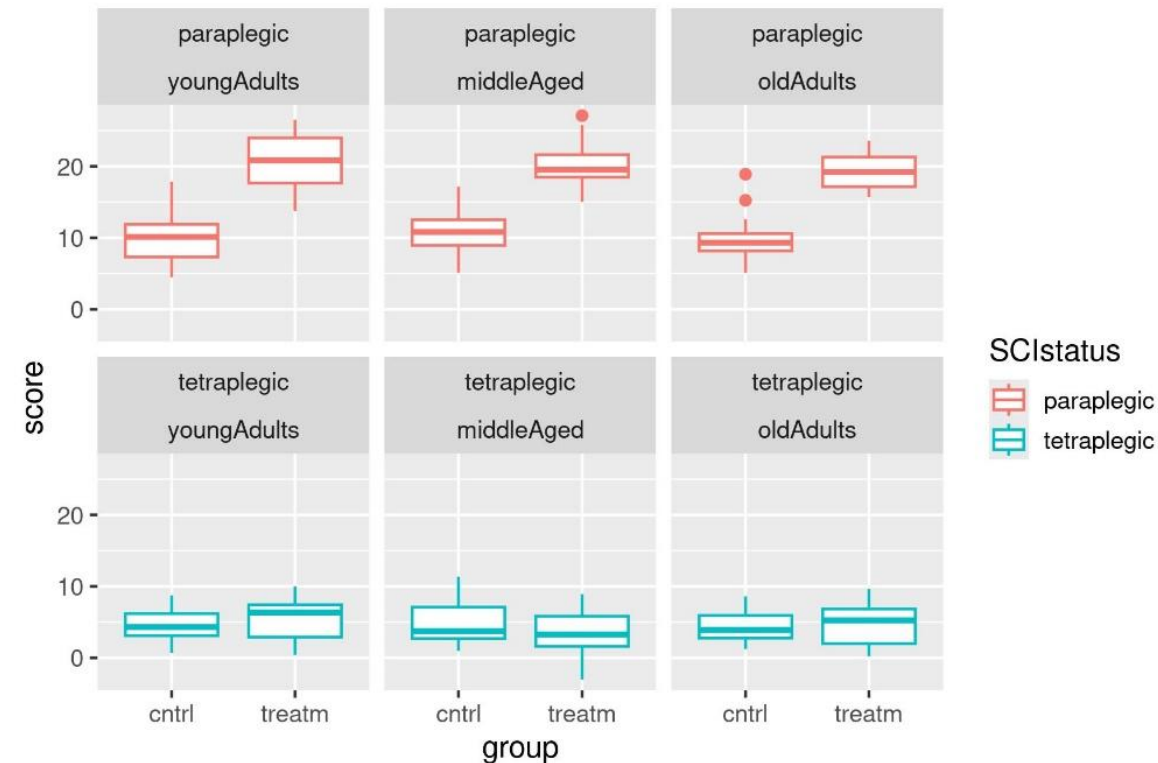
Creating figures with ggplot

- We can add new layers with the **+** sign
- ... Adding the **age** variable:

```
> ggplot(data = d1rg,  
  mapping = aes(x = group, y = score,  
                 colour = SCiStatus)) +  
  facet_wrap(~ SCiStatus + age) +  
  geom_boxplot()
```

```
> d1rg
```

	id	SCiStatus	group	age	score
1	pat1	tetraplegic	cntrl	oldAdults	2.86
2	pat2	paraplegic	treatm	youngAdults	20.89
3	pat3	paraplegic	cntrl	youngAdults	14.00
4	pat4	paraplegic	treatm	middleAged	18.46
5	pat5	paraplegic	cntrl	middleAged	7.53



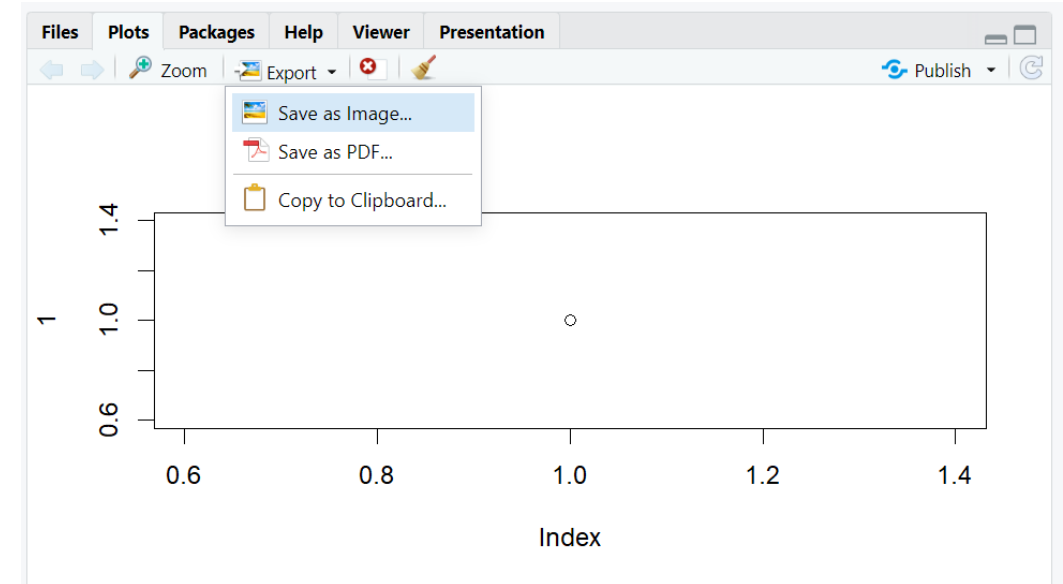
Exporting a figure

- In Rstudio we can export figures using the graphical interface
 - In the **Plots** tab > **Export** > **Save as Image/PDF**
- Alternatively, we can export figures using specific R commands
 - E.g. the **jpeg** function can be used to export a figure to a jpeg file (similar functions exist for png, bmp, pdf, ...):

```
> jpeg(filename = "myplot.jpg",  
       width = 10, height = 10,  
       units = "cm",  
       res = 300)
```

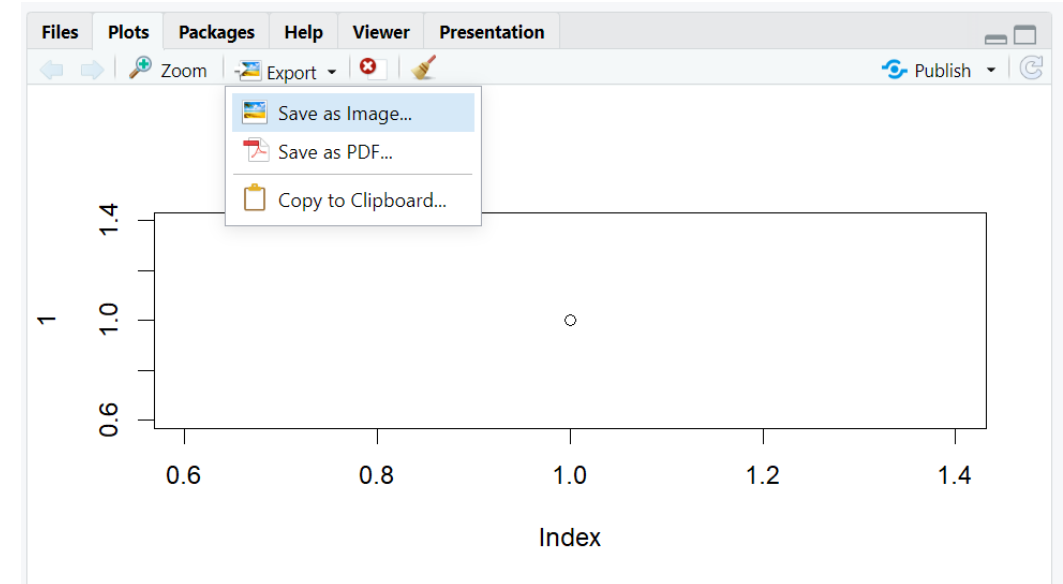
```
> plot(x = 1, y = 1)
```

```
> dev.off()
```



Exporting a figure

- In Rstudio we can export figures using the graphical interface
 - In the **Plots** tab > **Export** > **Save as Image/PDF**
- Alternatively, we can export figures using specific R commands
 - E.g. the **jpeg** function can be used to export a figure to a jpeg file (similar functions exist for png, bmp, pdf, ...):



```
> jpeg(filename = "myplot.jpg",  
       width = 10, height = 10,  
       units = "cm",  
       res = 300)
```

Start jpeg graphics device

```
> plot(x = 1, y = 1)
```

R code to create plot

```
> dev.off()
```

Stop graphics device

→ Image file will be saved
in working directory

Exercise: Data visualization

- Find the exercise at:

https://github.com/Swiss-Paraplegic-Research/Workshop-R-Introduction/tree/main/Part4_DataVisual/Exercise